Hardware Implementation of Neural Network Inference Using SystemVerilog - Phase 3: Combinational Neural Network Simulation Using Fixed-Point Parameters

1. Introduction

This report serves as a continuation of the last two phases and a starting point for the next phase, where we will implement a sequential Neural Network simulation in SystemVerilog using fixed-point parameters. In phase 1, a sequential MAC was designed to build the foundations of RTL. Section two focused on converting the retrieved floating-point parameters of a Shallow ANN in Python to fixed-point format. The objective of this phase is to simulate the Neural Network implemented in Python by building the combinational version of the Network in hardware.

This phase consists of four main sections.

Section one aims to select the appropriate signal width and discuss the signals' Q-format in this work.

Section two will focus on implementing the combinational network in SystemVerilog.

Section three focuses on the testbench, waveform analysis, and the network's performance on a sample.

Finally, in section four, a full analysis and comparison between Python and SystemVerilog results will be conducted. Furthermore, a conclusion regarding the choice of fixed-point ratio will be drawn that determines whether the chosen fixed-point ratio performed well, or any improvements can be employed in the future.

2. Fixed-Point Representation

The primary goal of this section is to choose the appropriate signal width and discuss the Qm.n format that has been derived from the analysis of the previous phase.

As discussed in phase 2, to achieve a suitable fixed-point format, i.e., compute the Qm.n, the maximum absolute value among the features in the training and testing sets, weighted sums, activations, and parameters has been computed. The achieved number then served as the threshold. The required number of **integer** bits, denoted as *m*, should be larger than this threshold.

The required number of **fractional** bits, denoted as *n*, was then computed by subtracting *n* from 16-bit fixed-point and *1 + m*:

$$1 + m + n = 16$$

It serves 12 binary bits for the fractional portion of the value.

A major difference between Python and SystemVerilog implementation in this project is the fact that all parameters utilized in SV are fixed-point versions of retrieved parameters from Python. Therefore, a 16-bit

signed fixed-point is suitable for this hardware project, since it does not support floating-point arithmetic. The number 1 in the formula above indicates a **sign-bit**, since the signals are a vector of signed numbers.

2.1 Fixed-Point Scaling

The neural network was implemented using the Q3.12 fixed-point format, where each value is represented as a signed 16-bit integer with 12 fractional bits. A stored integer value x_q corresponds to the real value:

$$x = \frac{X_q}{2^{12}}$$

Since $2^{12} = 4096$ the quantized representation of a real number is obtained by multiplying it by 4096 and rounding to the nearest integer.

2.2 Fixed-Point Multiplication

All inputs, weights, biases, hidden activations, and output activations are stored in Q3.12 format. When two Q3.12 values are multiplied, their scaling factors are multiplied as well:

$$(a \cdot 2^{12})(b \cdot 2^{12}) = (ab) \cdot 2^{24}$$

As a result, the multiplication output is no longer Q3.12. Instead, it contains 24 fractional bits and can be considered a Q6.24 value.

To return the product to the network's working format, the result must be shifted right by 12 bits:

$$Q6.24 \rightarrow Q3.12$$

To shift the result to the right by 12, the scaling operation `>>>` was used throughout the SV implementation. The right shift removes the extra 12 fractional bits introduced by the multiplication and restores the original Q3.12 scaling.

2.3 Fixed-Point Addition

Bias values are already stored in Q3.12 format. Once the multiplication result has been shifted back to Q3.12, both operands share the same scaling factor and can be added directly. No additional scaling is required after addition because addition does not change the fixed-point format.

For example:

6144 -> 1.5

$$9216 \text{ -> } 2.25$$

Adding these quantized values:

$$6144 + 9216 = 15360$$

The result remains a Q3.12 number because both operands were Q3.12 numbers. Converting back to floating point:

$$\frac{15360}{4096} = 3.75$$

Therefore, 15360 represents the value 3.75 in Q3.12 format.

An important question should be asked regarding the scaling: "*Where* is the Scaling required?"

The location of the scaling operation is determined by fixed-point arithmetic rules rather than experimental tuning.

Multiplication changes the number of fractional bits: Q3.12 × Q3.12 → Q6.24
while addition preserves the format: Q3.12 + Q3.12 → Q3.12

This explains why scaling is required after multiplication but not after addition. In fixed-point arithmetic, values should use the same scaling factor before addition.

2.4 Representable Range of Q3.12

There are mainly two crucial questions that should be taken into consideration regarding the Q3.12 range:

1. What is the accurate representable range of Q3.12?
2. What is the largest positive value that can be stored in Q3.12?

To address the first issue, it is important to note that a signed Q3.12 number uses:

- 1 sign bit
- 3 integer bits
- 12 fractional bits

The largest positive stored value is: $2^{15} - 1 = 32767$

which corresponds to: $\frac{32767}{4096} = 7.999755859375$

As discussed in the previous phase, the smallest representable value in this range is: -8.0

Thus, the accurately representable range of Q3.12 is approximately: [−8.0,7.9998]

Regarding the second question, the largest positive Q3.12 value, as shown above, is approximately 8.

Multiplying two maximum values produces: $7.9998 \times 7.9998 \approx 64$

In integer form:

$$32767 \times 32767 = 1{,}073{,}676{,}289$$

This intermediate result is represented in Q6.24 format. Converting it back to a real value:

$$\frac{1{,}073{,}676{,}289}{2^{24}} \approx 63.996$$

This demonstrates that multiplication can generate values far larger than those representable in the original 16-bit Q3.12 format. This brings us to the next important matter: “Importance of wider accumulators”

2.5 Overflow

Because multiplication temporarily increases both magnitude and precision, intermediate results must be stored in a wider accumulator. Using only a 16-bit accumulator would lead to overflow during multiply-accumulate operations.

For this reason, the hardware implementation used wider accumulator registers (a 32-bit vector result) to safely store intermediate products and partial sums before converting the final result back to Q3.12.

2.6 Signal Width

Below is a table of the signal width of this project’s network module:

Signal	**Width**	**Q-format**
x	16 bits	Q3.12
w_1	16 bits	Q3.12
z_1	32 bits	Q6.12
h	32 bits	Q6.12
w_2	16 bits	Q3.12
z_2	48 bits	Q9.12

Table 1. Signal Width and Q-format

Note that the resulting width of the z_2 equals 48 bits, namely, $M + N$.

As shown above, the Q-format of a signal is determined by the arithmetic operations and the location of the binary point, not by the width of the storage register. Increasing the register width increases the available numerical range and reduces the risk of overflow, but it does not change the number of fractional bits. In this design, multiplying two Q3.12 values produces a Q6.24 result. After shifting right by 12 bits, the result becomes Q6.12. Consequently, the hidden-layer outputs are represented as Q6.12 values even though they are stored in 32-bit accumulators. Similarly, the output-layer logits are represented as Q9.12 values while being stored in 48-bit accumulators to provide sufficient dynamic range and prevent overflow.

3. Combinational Network Implementation

The main objective of this section is to design a combinational shallow neural network with the architecture $2 \rightarrow 4 \rightarrow 2$ in SystemVerilog.

A shallow neural network consists of an input layer, a hidden layer, and an output layer. Likewise, a layer consists of several neurons. Each neuron is responsible for computing the weighted sum using a MAC, activating the neuron's weighted sum using ReLU, and making the final prediction by Argmax.

In a shallow neural network, there are mainly two layers that should be implemented. A *dense layer* and an *output layer*. Each of them is written in a separate module, composed of different components.

All of the modules have been written in separate `.sv` files to keep the code modularized. Each module is responsible for one task to ensure that the code follows the Single Responsibility Principle (SRP).

3.1 Parameterized Modules

To make the network reusable for future works with different architectures, every module has been implemented **parameterized**. A parameterized module can be configured by passing values when instantiating the module.

When defining a parameterized module, the entries within the parentheses preceded by a pound sign become the parameter list. Typically, parameters should be given names with all caps, as this makes it easy to distinguish which variables in the module definition are parameters and which are not. All parameters must also be given a default value; this is so that if a module is instantiated without changing a given parameter, there is a fallback value. *Adapted from [1]*

The general format for setting parameters when instantiating a module is below:

```
A_module #( .A_PARAMETER(a_constant_value), .ANOTHER_PARAMETER(7) ) an_instance_name
(
    .a_port(a_wire),
    .another_port(another_wire_or_constant)
)
```

3.2 Implementing Neuron Module

As explained above, a parameterized network is composed of layers, and the layers themselves are composed of neurons. To begin with, the neurons, as the foundational building blocks of the layers, should be implemented:

3.2.1 Neuron Module

```
module neuron #(
    parameter int N_INPUTS = 2,
                INPUT_WIDTH = 16,
                DATA_WIDTH = 16,
                ACC_WIDTH = 32
) (
```

The initial values of the parameters are set to the default network structure. The number of features has been set to two. Input and parameters are each stored in 16-bit vectors. The resulting width of the weighted sum is considered twice the inputs, i.e., 32 bits, to avoid **overflow**.

3.2.2 Neuron Module Ports

The ports (input and output signals) of the module `neuron` are shown below:

```
input logic signed [INPUT_WIDTH-1:0] x[N_INPUTS],
input logic signed [DATA_WIDTH-1:0] w[N_INPUTS],
input logic signed [DATA_WIDTH-1:0] b,
output logic signed [ACC_WIDTH-1:0] z
```

There are a few new concepts that should be introduced. First, the features are passed as an *array* of signed vectors. There are a total of two features, each with a width of 16 bits.

Similarly, weights are passed as an array of two elements. *The number of weights needed for each neuron is determined by the total number of features.* Since this work utilizes two features, the total number of weights needed for each neuron is two.

The two signals `b` and `z` are scalers, each of them holding one value.

3.2.3 Combinational MAC

The weighted sum (partial sum) can be computed using combinational MAC and for loop:

```
always_comb begin : linear_operation

    z = 0;

    for (int i = 0; i < N_INPUTS; i++) begin : MAC

        z += (x[i] * w[i]) >>> 12;

    end

    z += {{(ACC_WIDTH-DATA_WIDTH){b[DATA_WIDTH-1]}}, b};

end
```

MAC can be computed using combinational summation of multiple multiply-accumulate terms. Each input feature was multiplied by its associated weight in the loop and was shifted to the right by 12 before being passed to z. This shifting (scaling) ensures that the product will fall in the Q3.12 range before being added to z.

In the end, the bias is sign-extended to the accumulator width before addition. This line of code `{{(ACC_WIDTH-DATA_WIDTH){b[DATA_WIDTH-1]}}, b}` guarantees that the addition uses the intended signed representation and has the same width as the weighted sum.

3.3 Implementing Dense Layer

The module `dense_layer` is built using multiple instances of the `neuron` module. The next two dense layers, the hidden layer and the output layer, will then be built on top of this module.

```
module dense_layer #(
```

```
    parameter int N_INPUTS = 2,
                  N_NEURONS = 4,
                  INPUT_WIDTH = 16,
                  DATA_WIDTH = 16,
                  ACC_WIDTH = 32
) (
```

The module is parameterized as shown above. The parameters are set to the default value of the structure, i.e., $2 \rightarrow 4 \rightarrow 2$. The number of neurons is set to 4 for the hidden layer. Other parameters have the same value as before. The signals (parameters) will then be matched to the signals of the neuron module.

```
input logic signed [INPUT_WIDTH-1:0] x[N_INPUTS],
input logic signed [DATA_WIDTH-1:0] w[N_NEURONS][N_INPUTS],
input logic signed [DATA_WIDTH-1:0] b[N_NEURONS],
output logic signed [ACC_WIDTH-1:0] z[N_NEURONS]
```

Notably, the weights' signals in this module are *2-dimensional arrays*. The hidden-layer weight matrix is shown below:

$$W_1 = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \\ w_{41}^{(1)} & w_{42}^{(1)} \end{bmatrix}$$

Each row of the two-dimensional array is passed as the input signal to the `neuron` module. Each row consists of two 16-bit weights, each associated with one feature.
Similarly, each row of the bias and weighted sum arrays is passed along with the weights.

It is worth mentioning that the whole input array will be passed to the `neuron` module as the input signal.

3.3.1 Instantiating Neuron Module

To create multiple instances of a module in SystemVerilog, a type of loop is used, called a **generate loop**. A generate loop instantiates several copies of a hardware component and provides an easy and concise method to create multiple instances of module items, such as module instances. *Adapted from [2]*

The dense_layer's parameters are passed to the neuron's instances using the dot operator. The code is shown below:

```
generate
```

```
    for (genvar i = 0; i < N_NEURONS; i++) begin : Neurons

        neuron #(
            .N_INPUTS(N_INPUTS),
            .INPUT_WIDTH(INPUT_WIDTH),
            .DATA_WIDTH(DATA_WIDTH),
            .ACC_WIDTH(ACC_WIDTH)
            ) inst(
                .x(x),
                .w(w[i]),
                .b(b[i]),
                .z(z[i])
            );

    end

endgenerate
```

As shown above, the whole feature array has been passed, while for two-dimensional arrays, such as weights, biases, and weighted sums, each row of them was passed.

3.4 ReLU

The parameterized ReLU was implemented as follows:

```
module ReLU #(
    parameter int ACC_WIDTH = 32
) (

    input logic signed [ACC_WIDTH-1:0] in,
    output logic signed [ACC_WIDTH-1:0] out
);

    assign out = (in < 0) ? 0 : in;

endmodule
```

As mentioned in phase 1, ReLU does not change the width of the input signal. Therefore, the width of the output signal is the same as the width of the input signal as well.

ReLU is utilized between the hidden layer and the output layer. It activates the weighted sum and passes the activated values as the input to the output layer.

3.5 Argmax

Argmax is used after the output logits are computed to determine the predicted class. It makes the final prediction based on a comparison. If the first class's logit is equal to or higher than the second class's logit, the final prediction would be the first class; otherwise, the second class.

In code:

```
module Argmax #(
    parameter int ACC_WIDTH = 48
) (

    input logic signed [ACC_WIDTH-1:0] o0,
    input logic signed [ACC_WIDTH-1:0] o1,
    output logic pred
);

    assign pred = (o0 >= o1) ? 0 : 1;

endmodule
```

It is worth mentioning that the signal width of the inputs should be equal to the width of the computed output logits. Furthermore, `Argmax` will not be utilized in the output layer. It will be applied to the logits, after they are computed.

3.6 Network Module

The network module is composed of all the modules built so far:

```
module network #(
    parameter int N_INPUTS = 2,
                  N_NEURONS = 4,
                  N_OUTPUTS = 2,
                  DATA_WIDTH = 16,
                  ACC1_WIDTH = 32,
                  ACC2_WIDTH = 48
```

```
) (
```

The ports of the Network module will be matched to the hidden_layer and output_layer, which are built upon dense_layer.

```
input logic signed [DATA_WIDTH-1:0] x[N_INPUTS],
input logic signed [DATA_WIDTH-1:0] w1[N_NEURONS][N_INPUTS],
input logic signed [DATA_WIDTH-1:0] b1[N_NEURONS],

input logic signed [DATA_WIDTH-1:0] w2[N_OUTPUTS][N_NEURONS],
input logic signed [DATA_WIDTH-1:0] b2[N_OUTPUTS],

output logic signed [ACC2_WIDTH-1:0] z2[N_OUTPUTS],
output logic pred_class
```

The two weight matrices were declared as two-dimensional arrays. The weight matrix of the output layer is in the following architecture:

$$W_2 = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} & w_{14}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} & w_{24}^{(2)} \end{bmatrix}$$

The two helper signals z_1 and h were declared as follows:

```
// initialize the layers' helper signals
logic signed [ACC1_WIDTH-1:0] z1[N_NEURONS];
logic signed [ACC1_WIDTH-1:0] h[N_NEURONS];
```

3.6.1 Hidden Layer

```
// build the first hidden layer
dense_layer #(
    .N_INPUTS(N_INPUTS),
    .N_NEURONS(N_NEURONS),
    .INPUT_WIDTH(DATA_WIDTH),
    .DATA_WIDTH(DATA_WIDTH),
    .ACC_WIDTH(ACC1_WIDTH)
    ) hidden_layer(
        .x(x),
```

```
        .w(w1),
        .b(b1),
        .z(z1)
    );
```

```
// pass the scaled signals to ReLU
generate

    for (genvar i = 0; i < N_NEURONS; i++) begin : ReLU_Activation

        ReLU #(
            .ACC_WIDTH(ACC1_WIDTH)
            ) activate(
                .in(z1[i]),
                .out(h[i])
        );

    end

endgenerate
```

Note that a generate loop was utilized to create four hardware copies of the ReLU module.

3.6.2 Output Layer

```
// build the output layer using the previous layer's activations and return the
logits
dense_layer #(
    .N_INPUTS(N_NEURONS),
    .N_NEURONS(N_OUTPUTS),
    .INPUT_WIDTH(ACC1_WIDTH),
    .DATA_WIDTH(DATA_WIDTH),
    .ACC_WIDTH(ACC2_WIDTH)
    ) output_layer(
        .x(h),
        .w(w2),
        .b(b2),
        .z(z2)
);
```

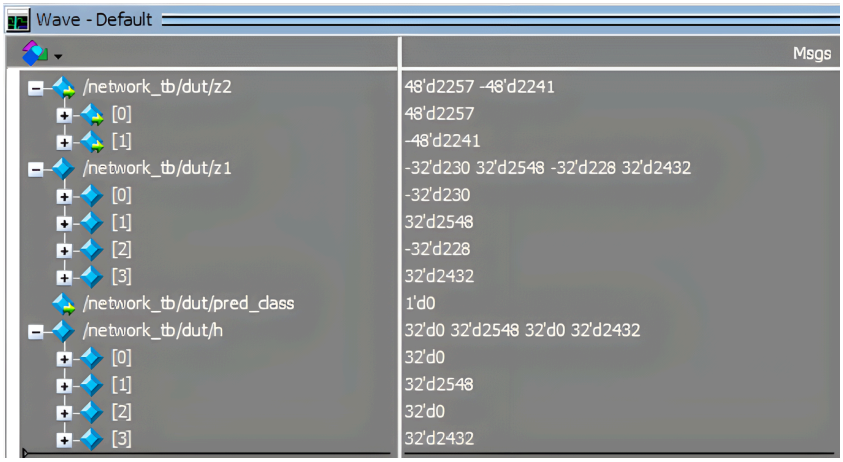
3.6.3 Final Prediction

```
// predict the class by returning the maximum logit
Argmax #(
    .ACC_WIDTH(ACC2_WIDTH)
    ) predict(
        .o0(z2[0]),
        .o1(z2[1]),
        .pred(pred_class)
);
```

4. Network Testbench

The quantized test set's features and the trained parameters retrieved from Python have been used for testing the Network module performance in QuestaSim.

A waveform of a sample with X_test values of [579 -1200] is provided below:



The numbers are in Decimal. The first number, written before the letter d, which stands for decimal, demonstrates the width of the signal, while the value afterwards showcases the predictions by the network. For instance, z_2 holds two values as expected. The first element of the array is 2257, and the second element is -2241. The final prediction, as the `pred_class` predicted, is a one-bit signal of class 0.

This matches the prediction and ground truth from Python as well:

In Python:

```
Sample 0 X = [579 -1200] z1 = [-230 2548 -228 2432]  a1 = [   0 2548    0
2432]  z2 = [ 2257 -2241]  pred = 0
```

The first 10 predictions in SV and Python are provided in the next section.

5. Final Results

A comparison between the last 10 predictions in SystemVerilog and Python:

Sample #	Prediction	Ground Truth
47	1	1
48	1	1
49	1	1
50	0	0
51	0	0
52	0	0
53	1	1
54	0	0
55	1	1
56	0	1

Table 2. Python Predictions

Sample #	Prediction	Ground Truth
47	1	1
48	1	1
49	1	1
50	0	0
51	0	0
52	0	0
53	1	1
54	0	0
55	1	1
56	0	1

Table 3. SystemVerilog Predictions

Overall, the SystemVerilog implementation produced identical predictions to the Python fixed-point reference for all 57 samples, corresponding to 100% agreement between the two implementations.

Summary

This report continued the work of building a combinational Neural Network simulation in SystemVerilog using fixed-point parameters. The appropriate signal widths and Q-format for the network have been selected, and the Network was implemented in SystemVerilog. The testbench and the network's performance have also been analyzed. Important topics such as fixed-point representation, scaling techniques, and the differences between multiplication and addition in fixed-point arithmetic were

covered. Finally, conclusions were drawn by comparing the results from the Python and SystemVerilog implementations, which provided insights for improving fixed-point performance in the future.

References

---

[1] “Parameterized Verilog Modules,” *Real Digital*. [Online]. Available: https://www.realdigital.org/doc/43c79714a7f3d0bbb8098d60c63fde48. [Accessed: Jun. 20, 2026].

[2] “SystemVerilog Generate Construct,” systemverilog.io. [Online]. Available: https://www.systemverilog.io/verification/generate/. [Accessed: Jun. 20, 2026].

By Kiana Jafari - Computer Engineering Student